# PROYECTO FINAL: SISTEMA DE IA INTEGRAL

**Entrega Final, Video Pitch y Video de Preguntas**

---

## DESCRIPCIÓN GENERAL

El proyecto final representa la culminación de todo el trabajo realizado durante el semestre en el curso de Inteligencia Artificial. Deberán entregar un **sistema de IA completo y funcional** que integre todos los conocimientos adquiridos: desde la planificación inicial, desarrollo del modelo, optimización, hasta consideraciones éticas y despliegue con interfaz de usuario.

Este proyecto debe demostrar su capacidad para:

- Resolver un problema real usando técnicas de IA
- Aplicar metodologías profesionales de desarrollo
- Implementar buenas prácticas de documentación y código
- Considerar aspectos éticos y sociales de la IA
- Desplegar una solución accesible y usable

---

## OBJETIVOS DEL PROYECTO FINAL

1. **Integración completa:** Demostrar dominio de todo el ciclo de vida de un proyecto de IA
2. **Implementación funcional:** Entregar un sistema que realmente funcione y sea utilizable
3. **Profesionalismo:** Seguir estándares de la industria en documentación y código
4. **Impacto:** Resolver un problema con relevancia real y consideraciones éticas
5. **Comunicación:** Presentar el proyecto de manera clara y convincente

---

## COMPONENTES OBLIGATORIOS DE LA ENTREGA

### 1. REPOSITORIO DE GITHUB (60% de la calificación)

El proyecto completo debe estar alojado en un repositorio público de GitHub con la siguiente estructura:

#### Estructura de carpetas requerida:

```
nombre-proyecto/
│
├── README.md                      # Descripción principal del proyecto
├── requirements.txt               # Dependencias de Python
├── .gitignore                     # Archivos a ignorar
├── LICENSE                        # Licencia del proyecto
│
├── docs/                          # Documentación
│   ├── planificacion.md           # Documento de planificación inicial
│   ├── analisis_datos.md          # Análisis exploratorio de datos
│   ├── arquitectura.md            # Diseño del modelo y arquitectura
│   ├── optimizacion.md            # Proceso de optimización (Workshop S5)
│   ├── consideraciones_eticas.md  # Análisis ético del proyecto
│   └── manual_usuario.md          # Guía para usuarios finales
│
```

```
├── data/                          # Datos del proyecto
│   ├── raw/                       # Datos originales (si aplica)
│   ├── processed/                 # Datos procesados
│   └── README.md                  # Descripción de los datos
│
├── notebooks/                     # Jupyter notebooks
│   ├── 01_exploracion.ipynb       # EDA
│   ├── 02_preprocesamiento.ipynb  # Limpieza y preparación
│   ├── 03_modelado.ipynb          # Entrenamiento de modelos
│   ├── 04_optimizacion.ipynb      # Optimización de hiperparámetros
│   └── 05_evaluacion.ipynb        # Evaluación final
│
├── src/                           # Código fuente
│   ├── __init__.py
│   ├── data_processing.py         # Funciones de procesamiento
│   ├── model.py                   # Definición del modelo
│   ├── train.py                   # Script de entrenamiento
│   ├── evaluate.py                # Script de evaluación
│   └── utils.py                   # Funciones auxiliares
│
├── models/                        # Modelos entrenados
│   ├── best_model.pkl             # Mejor modelo guardado
│   ├── model_v1.pkl               # Versiones anteriores
│   └── README.md                  # Descripción de modelos
│
├── app/                           # Aplicación/Interfaz
│   ├── app.py                     # Aplicación principal (Streamlit/Gradio/Flask)
│   ├── requirements.txt           # Dependencias de la app
│   └── assets/                    # Recursos (imágenes, CSS, etc.)
│
├── tests/                         # Pruebas unitarias
│   ├── test_data_processing.py
│   ├── test_model.py
│   └── test_app.py
│
└── results/                       # Resultados y visualizaciones
    ├── figures/                   # Gráficos generados
    ├── metrics/                   # Métricas de evaluación
    └── reports/                   # Reportes generados
```

**Contenido obligatorio del README.md principal:**

El archivo README.md es la carta de presentación del proyecto. Debe incluir:

**1. Encabezado y badges:**

- Título del proyecto
- Badges de estado (Python version, License, etc.)
- Descripción breve (2-3 oraciones)

**2. Tabla de contenidos**

**3. Descripción del problema:**

- ¿Qué problema resuelve el proyecto?
- ¿Por qué es importante?
- ¿Quiénes son los usuarios objetivo?

**4. Dataset:**

- Descripción de los datos utilizados

- Fuente y licencia de los datos
- Características principales (tamaño, variables, etc.)
- Link a los datos si son públicos

**5. Metodología:**

- Tipo de modelo utilizado y justificación
- Preprocesamiento aplicado
- Técnicas de optimización empleadas
- Métricas de evaluación seleccionadas

**6. Resultados:**

- Métricas finales del modelo (accuracy, F1, RMSE, etc.)
- Comparación con baseline
- Tabla o gráfico de rendimiento

**7. Instalación y uso:**

- Requisitos del sistema
- Instrucciones paso a paso para instalar
- Comandos para ejecutar el proyecto
- Ejemplos de uso

**8. Interfaz de usuario:**

- Descripción de la interfaz
- Capturas de pantalla
- Link a la aplicación desplegada (si aplica)
- Instrucciones para usar la interfaz

**9. Estructura del proyecto:**

- Descripción breve de cada carpeta principal

**10. Consideraciones éticas:**

- Resumen de aspectos éticos considerados
- Limitaciones conocidas del modelo
- Advertencias sobre uso inadecuado

**11. Autores y contribuciones:**

- Nombres de los integrantes del equipo
- Rol de cada integrante

**12. Licencia**

**13. Agradecimientos y referencias**

**14. Videos:**

- Link al video pitch (5 minutos)
- Link al video de respuestas a las preguntas

**2. INTERFAZ DE USUARIO FUNCIONAL (20% de la calificación)**

Deben implementar una interfaz que permita a cualquier persona probar su modelo de IA de forma interactiva.

**Opciones de tecnología (elegir UNA):**

**Opción A: Streamlit (Recomendado - Más fácil)**

- Framework de Python para crear aplicaciones web de ML rápidamente
- Ideal para prototipos y demos
- No requiere conocimientos de HTML/CSS/JavaScript

**Opción B: Gradio**

- Similar a Streamlit, especializado en ML
- Excelente para crear interfaces de demostración
- Fácil integración con Hugging Face Spaces

**Opción C: Flask + HTML/CSS**

- Más control sobre el diseño
- Requiere conocimientos de desarrollo web
- Recomendado solo si tienen experiencia previa

**Opción D: FastAPI + Frontend (React/Vue)**

- Opción más avanzada y profesional
- Para equipos con experiencia en desarrollo web
- Permite arquitecturas más escalables

**Requisitos mínimos de la interfaz:**

**Funcionalidades obligatorias:**

**1. Input de datos:**

- Formulario o método para que el usuario ingrese datos
- Puede ser: texto, imagen, archivo CSV, valores numéricos, etc.
- Validación de inputs (verificar que los datos sean correctos)

**2. Procesamiento:**

- Botón o acción para ejecutar la predicción
- Indicador de carga mientras procesa
- Manejo de errores (mensajes si algo sale mal)

**3. Output/Resultados:**

- Mostrar la predicción del modelo de forma clara
- Incluir nivel de confianza o probabilidades
- Visualizaciones relevantes (gráficos, tablas, etc.)

**4. Información del modelo:**

- Sección "Acerca de" con descripción del modelo
- Métricas de rendimiento
- Limitaciones y advertencias

**5. Ejemplos de uso:**

- Datos de ejemplo precargados que los usuarios puedan probar
- Botón "Prueba con ejemplo" que carga datos automáticamente

**Elementos adicionales valorados:**

- Diseño responsive (funciona en móvil y escritorio)
- Temas claros/oscuros
- Múltiples idiomas (español e inglés)
- Exportación de resultados (descargar predicción en PDF/CSV)
- Historial de predicciones en la sesión
- Comparación de múltiples predicciones
- Visualizaciones interactivas (plotly, altair)

**Criterios de evaluación de la interfaz:**

- **Funcionalidad:** Hace lo que promete sin errores
- **Usabilidad:** Es intuitiva y fácil de usar
- **Diseño:** Interfaz limpia y profesional
- **Robustez:** Maneja errores de forma elegante
- **Documentación:** Instrucciones claras dentro de la app

**3. DOCUMENTACIÓN COMPLETA EN /docs (10% de la calificación)**

Deben incluir los siguientes documentos en la carpeta docs/:

**A) planificacion.md**
**Contenido:**

- Definición del problema y objetivos
- Justificación de la relevancia del proyecto
- Alcance (qué incluye y qué NO incluye)
- Cronograma de desarrollo (planificado vs real)
- Recursos necesarios (datos, hardware, software)
- Riesgos identificados y mitigación

**B) analisis_datos.md**
**Contenido:**

- Descripción detallada del dataset
- Estadísticas descriptivas
- Visualizaciones del EDA (mínimo 5-6 gráficos relevantes)
- Identificación de patrones, correlaciones, outliers
- Decisiones de preprocesamiento justificadas
- Manejo de datos faltantes o desbalanceados

**C) arquitectura.md**
**Contenido:**

- Tipo de modelo seleccionado y justificación
- Arquitectura detallada (capas, parámetros, etc.)
- Diagrama de flujo del sistema completo
- Pipeline de datos (desde input hasta output)
- Tecnologías y librerías utilizadas con versiones

**D) optimizacion.md**
**Contenido:**

- Proceso de optimización de hiperparámetros (del Workshop S5)
- Hiperparámetros explorados y rangos
- Resultados del análisis de sensibilidad
- Partial dependence plots
- Ranking de importancia de hiperparámetros
- Análisis de interacciones
- Configuración final seleccionada y justificación
- Comparación antes/después de la optimización

**E) consideraciones_eticas.md** **CRÍTICO**
**Contenido obligatorio:**

**1. Análisis de sesgos:**
- ¿El dataset tiene sesgos demográficos, culturales o de otro tipo?
- ¿Cómo podrían afectar estos sesgos las predicciones?
- ¿Qué grupos podrían ser perjudicados?

**2. Equidad y fairness:**
- ¿El modelo trata a todos los grupos de forma equitativa?
- Métricas de fairness evaluadas (si aplica)
- Estrategias implementadas para mitigar inequidades

**3. Privacidad:**
- ¿Se utilizan datos personales o sensibles?
- ¿Cómo se protege la privacidad de los usuarios?
- Cumplimiento con regulaciones (GDPR, CCPA, etc.)

**4. Transparencia y explicabilidad:**
- ¿El modelo es interpretable?
- ¿Los usuarios entienden cómo funciona?
- Técnicas de explicabilidad implementadas (SHAP, LIME, etc.)

**5. Impacto social:**
- ¿Qué impacto positivo puede tener el proyecto?
- ¿Qué impactos negativos podrían surgir?
- ¿Quiénes se benefician? ¿Quiénes podrían ser perjudicados?

**6. Responsabilidad:**
- ¿Quién es responsable si el modelo falla?
- ¿Qué mecanismos de accountability existen?
- Plan de monitoreo y actualización del modelo

**7. Uso dual y mal uso:**
- ¿Podría el modelo usarse con fines maliciosos?
- ¿Qué salvaguardas se han implementado?
- Limitaciones de uso claramente documentadas

**8. Limitaciones reconocidas:**

- ¿En qué casos NO debe usarse el modelo?
- ¿Qué advertencias deben darse a los usuarios?
- Casos límite donde el modelo no es confiable

**F) manual_usuario.md**
**Contenido:**

- Guía paso a paso para usar la interfaz
- Capturas de pantalla anotadas
- Explicación de cada funcionalidad
- Troubleshooting (problemas comunes y soluciones)
- FAQ (preguntas frecuentes)
- Información de contacto para soporte

**4. CÓDIGO FUENTE DE CALIDAD (10% de la calificación)**

**Requisitos de calidad del código:**

**A) Estilo y legibilidad:**

- Seguir PEP 8 (estándar de Python)
- Nombres de variables descriptivos
- Funciones y clases bien documentadas con docstrings
- Código comentado en secciones complejas
- Consistencia en nomenclatura

**B) Modularidad:**

- Código organizado en funciones y clases
- Separación de responsabilidades (procesamiento, modelo, evaluación)
- Reutilización de código (DRY - Don't Repeat Yourself)
- Importaciones organizadas

**C) Manejo de errores:**

- Try-except en operaciones críticas
- Mensajes de error informativos
- Validación de inputs
- Logging de eventos importantes

**D) Configuración:**

- Parámetros configurables (no hardcodeados)
- Uso de archivos de configuración (config.yaml, .env)
- Rutas relativas (no absolutas)

**E) Versionamiento:**

- Commits frecuentes con mensajes descriptivos
- Uso de branches para desarrollo
- Tags para versiones importantes
- .gitignore apropiado

**F) Dependencias:**

- requirements.txt completo y actualizado
- Versiones específicas de librerías
- Instrucciones de instalación claras

**G) Pruebas:**

- Al menos 3-5 pruebas unitarias básicas
- Pruebas de funciones críticas
- Documentación de cómo ejecutar las pruebas

---

## PRESENTACIÓN FINAL: VIDEO PITCH Y VIDEO DE PREGUNTAS (Obligatoria)

La presentación final **no se realiza en vivo** y **no incluye demo en vivo**. En su lugar, cada equipo debe entregar **dos videos obligatorios**:

1. **Video pitch** de 5 minutos que presenta el proyecto completo.
2. **Video de respuestas** a preguntas puntuales sobre su proyecto, que serán repartidas por el profesor en la última semana de clases.

### A) VIDEO PITCH (5 minutos)

**Duración:** máximo 5 minutos (límite estricto).

El pitch debe convencer a la audiencia de que su sistema resuelve un problema real, funciona y ha sido desarrollado de forma rigurosa y responsable. La interfaz se muestra mediante una **grabación de pantalla** incluida dentro del video.

**Estructura obligatoria:**

**1. Introducción y problema (30 segundos)**

- Nombre del proyecto y equipo
- Problema que resuelven
- Motivación y relevancia

**2. Datos y Metodología (1 minuto)**

- Dataset utilizado
- Preprocesamiento realizado
- Tipo de modelo y arquitectura
- Justificación de decisiones técnicas

**3. Desarrollo y Optimización (45 segundos)**

- Desafíos principales encontrados y soluciones
- Técnicas de optimización aplicadas
- Hallazgos clave del análisis de sensibilidad de hiperparámetros

**4. La solución en acción (1 minuto)** **CRÍTICO**

- Grabación de pantalla de la interfaz funcionando realmente
- Mostrar al menos 2-3 casos de uso diferentes
- Explicar outputs y visualizaciones
- Mostrar manejo de errores

**5. Evaluación y Resultados (45 segundos)**

- Métricas de rendimiento
- Comparación con baseline o estado del arte
- Casos donde funciona bien y donde no

**6. Consideraciones Éticas (45 segundos)** **OBLIGATORIO**

- Sesgos identificados
- Impacto social potencial
- Medidas de mitigación implementadas
- Limitaciones éticas reconocidas

**7. Cierre (15 segundos)**

- Logro principal y trabajo futuro
- Link al repositorio GitHub y a la aplicación (con QR code en pantalla)

**Requisitos técnicos del video pitch:**

**Diapositivas / material visual:**

- Máximo 8-10 slides (adaptado a 5 minutos)
- Diseño profesional y consistente
- Gráficos y visualizaciones de alta calidad
- Poco texto, más imágenes y diagramas
- Código solo si es absolutamente necesario (máximo 5-10 líneas)

**Grabación:**

- Audio claro y sin ruido de fondo; imagen nítida (mínimo 720p)
- La interfaz mostrada debe ser la versión real entregada en el repositorio (no maquetas ni simulaciones)
- Explicación clara de lo que se está mostrando en pantalla
- Formato MP4, o link a YouTube (no listado) / Google Drive con acceso habilitado para el profesor

**Material de apoyo:**

- Link al repositorio GitHub visible en el video
- QR code para acceso rápido al proyecto
- Handout opcional con resumen del proyecto

**B) VIDEO DE RESPUESTAS A PREGUNTAS**

Este video reemplaza la sesión de preguntas en vivo. Durante la **última semana de clases**, el profesor entregará a cada equipo un conjunto de **preguntas puntuales sobre su propio proyecto** (decisiones técnicas, resultados, optimización, interfaz, ética, limitaciones, etc.).

**Requisitos:**

- Responder todas las preguntas asignadas, en el orden en que fueron entregadas
- Indicar el número de la pregunta antes de cada respuesta
- Responder de forma directa, concreta y técnicamente fundamentada
- Defender las decisiones técnicas tomadas y discutir abiertamente las limitaciones

- Apoyarse en el código, los resultados o la interfaz cuando la pregunta lo requiera (grabación de pantalla permitida)
- Duración y plazo de entrega: según se indique junto con las preguntas

**Entrega final:**

- Repositorio GitHub completo y pulido
- Interfaz completamente funcional
- Toda la documentación completa
- **Video pitch de 5 minutos (obligatorio)**
- **Video de respuestas a las preguntas (obligatorio)**
- Links a ambos videos incluidos en el README

**Fechas:**

- Entrega de preguntas a cada equipo: última semana de clases
- Fecha límite de entrega del video pitch: _____
- Fecha límite de entrega del video de preguntas: _____

---

## RÚBRICA DE EVALUACIÓN DETALLADA

| Criterio | Excelente (9-10) | Bueno (7-8) | Suficiente (5-6) | Insuficiente (<5) | Peso |
|---|---|---|---|---|---|
| **Repositorio GitHub - Estructura** | Estructura perfecta, todos los archivos organizados, README excepcional | Estructura correcta, README completo | Estructura básica, README mínimo | Desorganizado, README incompleto | 10% |
| **Repositorio - Documentación** | Todos los docs completos, detallados y profesionales | Mayoría de docs completos y claros | Docs básicos presentes | Docs faltantes o muy superficiales | 15% |
| **Código - Calidad** | Código limpio, modular, bien documentado, sigue PEP8, incluye tests | Código funcional, organizado, documentado | Código funcional pero desorganizado | Código con errores o muy pobre | 10% |
| **Modelo - Desarrollo** | Modelo sofisticado, bien justificado, optimizado profesionalmente | Modelo apropiado, optimización aplicada | Modelo básico funcional | Modelo inadecuado o no funciona | 15% |
| **Optimización de Hiperparámetros** | Análisis completo (Workshop S5), mejoras significativas documentadas | Análisis correcto, algunas mejoras | Optimización básica aplicada | Sin optimización o mal hecha | 10% |
| **Interfaz - Funcionalidad** | Interfaz profesional, robusta, sin errores, excelente UX | Interfaz funcional, buen diseño, pocos errores | Interfaz básica que funciona | Interfaz con errores o no funciona | 10% |
| **Interfaz - Usabilidad** | Intuitiva, atractiva, ejemplos claros, bien documentada | Fácil de usar, documentación adecuada | Usable con esfuerzo | Confusa o difícil de usar | 5% |

| Criterio | Excelente (9-10) | Bueno (7-8) | Suficiente (5-6) | Insuficiente (<5) | Peso |
|---|---|---|---|---|---|
| **Consideraciones Éticas** | Análisis profundo, múltiples aspectos, mitigaciones implementadas | Análisis completo de aspectos principales | Consideraciones básicas presentes | Análisis superficial o ausente | 10% |
| **Evaluación y Resultados** | Evaluación rigurosa, múltiples métricas, análisis de errores detallado | Evaluación correcta con métricas apropiadas | Evaluación básica presente | Evaluación inadecuada o faltante | 10% |
| **Presentación (video pitch y video de preguntas)** | Pitch claro y convincente, interfaz mostrada impecablemente, respuestas precisas y bien fundamentadas | Buen pitch, interfaz se muestra funcionando bien, respuestas correctas | Pitch aceptable, interfaz con fallas menores, respuestas incompletas | Pitch pobre, interfaz no funciona o no se muestra, respuestas ausentes o incorrectas | Requisito |

**Total: 100 puntos**

---

## CRITERIOS DE EXCELENCIA (Para obtener 9-10)

Un proyecto excelente debe cumplir con:

- **Innovación:** Aborda el problema de forma original o creativa
- **Completitud:** Todos los componentes presentes y bien ejecutados
- **Profesionalismo:** Calidad que podría presentarse en la industria
- **Impacto:** Solución real y útil para un problema relevante
- **Ética:** Consideraciones éticas profundas y bien integradas
- **Documentación excepcional:** README y docs que facilitan replicación
- **Interfaz pulida:** UX profesional y agradable de usar
- **Optimización demostrada:** Mejoras cuantificables del modelo
- **Presentación destacada:** Pitch claro y convincente, y respuestas sólidas a las preguntas
- **Reproducibilidad:** Otro desarrollador puede ejecutar el proyecto fácilmente

---

## CAUSALES DE PENALIZACIÓN O RECHAZO

**Penalizaciones:**

- -10%: Entrega tardía (por cada día de retraso)
- -15%: Repositorio privado o inaccesible
- -20%: Interfaz no funcional o con errores críticos
- -20%: Documentación ética ausente o superficial
- -10%: Código plagiado de internet sin atribución
- -15%: No entregar el video pitch o el video de preguntas en la fecha asignada (sin justificación)

**Causales de rechazo automático (calificación 0):**

- Proyecto plagiado completamente de otro equipo o fuente

- Modelo que no funciona en absoluto
- Repositorio falso o datos fabricados
- No entregar los videos sin justificación médica o de fuerza mayor
- Violaciones éticas graves (datasets ilegales, aplicaciones dañinas, etc.)

---

## RECOMENDACIONES PARA EL ÉXITO

**Planificación:**

- Empiecen temprano, no dejen todo para última semana
- Dividan tareas claramente entre miembros del equipo
- Usen GitHub Projects o Trello para gestión de tareas
- Hagan commits frecuentes y pequeños

**Desarrollo:**

- Prueben la interfaz con usuarios reales (compañeros, familia)
- Documenten a medida que avanzan, no al final
- Hagan respaldos frecuentes del código y modelos
- Pidan retroalimentación temprana al profesor

**Documentación:**

- Escriban para una audiencia técnica pero no experta
- Usen diagramas y visualizaciones, no solo texto
- Revisen ortografía y gramática
- Incluyan referencias y créditos apropiados

**Interfaz:**

- Prueben en diferentes navegadores y dispositivos
- Optimicen tiempos de carga
- Incluyan mensajes de ayuda y tooltips
- Manejo elegante de errores (no mensajes técnicos para usuarios)

**Ética:**

- Sean honestos sobre limitaciones
- No exageren capacidades del modelo
- Consideren perspectivas diversas
- Consulten con expertos en ética si es necesario

**Videos:**

- Escriban un guion para el pitch y ensáyenlo múltiples veces
- Cronometren: el pitch no debe superar los 5 minutos
- Graben la interfaz funcionando varias veces y elijan la mejor toma
- Revisen audio, imagen y links antes de entregar
- Preparen desde antes respuestas a preguntas probables sobre su proyecto

## RECURSOS Y REFERENCIAS

**Herramientas recomendadas:**

**Para interfaz:**

- Streamlit: https://streamlit.io/
- Gradio: https://gradio.app/
- Flask: https://flask.palletsprojects.com/

**Para documentación:**

- MkDocs: https://www.mkdocs.org/
- Sphinx: https://www.sphinx-doc.org/
- GitHub Pages

**Para visualización:**

- Plotly: https://plotly.com/python/
- Matplotlib/Seaborn
- Altair

**Para ética en IA:**

- AI Ethics Guidelines: https://ethics.fast.ai/
- Fairlearn: https://fairlearn.org/
- Google AI Principles